Model Context Protocol (MCP)

Revised Notes: concept clear + interview ready (Roman Urdu)

Is document mein kya hai

Ye notes 3 sources ko merge kar ke bane hain: (1) tumhara purana MCP_Complete_Notes.pdf, (2) handwritten class notes, (3) class ka poora transcript (theory + code demo + doubt session). Repeated points ek jagah kar diye hain, aur jo concepts missing thay (Tools/Resources/Prompts, URI, Host vs Client, packages ka farq, async ki wajah, scaling, vendor MCP servers, name conflict, A2A, interview Q&A) add kiye hain.

Roadmap

Part	Topic	Kyun zaroori hai
1	Problem: MCP ke bina kya masla	Interview ka pehla sawal: 'MCP kyun?'
2	Solution + analogies (USB-C, Browser/Server)	Concept dimagh mein baithane ke liye
3	Host / Client / Server aur packages ka farq	MCP vs FastMCP vs LangChain adapter
4	Server ki 3 cheezen: Tools, Resources, Prompts	Sabse zyada confuse karne wala hissa
5	Transport (STDIO / HTTP) + JSON-RPC	Local vs remote, message format
6	Code walkthrough: server.py, client.py, agent client	Practical samajh
7	Real-world: scaling, vendor servers, security, A2A	Senior-level follow-up questions
8	MCP vs normal function calling	Common interview comparison
9	Interview Q&A + cheat sheet	Last-minute revision

Yaad rakho

Ek line mein MCP: MCP ek open standard protocol hai jo AI applications ko external systems (DB, GitHub, Calendar, Files, APIs) se **ek standard tareeqe** se connect karta hai, taake har app aur har system ke liye alag custom integration na likhni paray.

Part 1: MCP ke bina kya masla tha?

1.1 Normal AI app

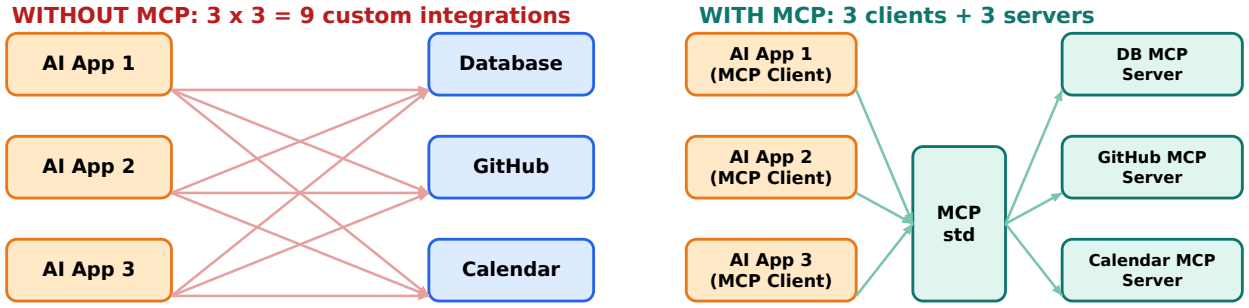
User sawal deta hai, AI app usay LLM ko deta hai, LLM jawab de deta hai. **LLM app ka dimagh hai**, lekin akela LLM sirf apni training knowledge se jawab deta hai.

1.2 Jab LLM ko external system chahiye

Maan lo app ko Database, GitHub, Google Calendar, Slack ya files se info chahiye. **LLM khud se in systems tak magically access nahi kar sakta.** Humein integration likhni parti hai, aur ye integration sirf ek **custom code / script** hota hai jo us system se baat karta hai (jaise Postgres query, GitHub API call).

1.3 N x M problem (sabse important picture)

Ek app aur 5 systems = 5 alag integrations banani aur maintain karni parti hain. Company mein ek nahi, kai AI apps hoti hain (Claude Chat, Cursor, custom apps), aur har app ko har system ke liye apni integration chahiye. Integrations multiply hoti jati hain: **N apps x M systems**.



1.4 Is approach ke 3 bade nuksan

- **Duplication:** same Database/GitHub integration har app mein dobara likhni parti hai.
- **API version change:** company API v1 se v2 kar de to aapki app break ho jati hai, code khol kar manually fix karna paray ga. Hazaron tools hon to har API ki documentation roz check karna namumkin hai.
- **Framework lock-in:** LangGraph mein tool likha, kal CrewAI / OpenAI Agent SDK / Agno par jana ho to wahi tool dobara likhna parta hai.

> **Result**
>
> Yahi wo wajah hai jis se MCP introduce hua: **tool calling ko standardize** karna, taake tool ek jagah (server par) ban jaye aur har app use kar sake.

Part 2: MCP kya hai? (Solution)

MCP = Model Context Protocol. Naam ko todo: **Model** = LLM, **Context** = wo info jo LLM ko chahiye (external systems se), **Protocol** = connect karne ke fixed rules (client-server architecture, STDIO ya HTTP transport).

2.1 Kis ne banaya?

- **Anthropic** ne MCP ko **open-source project** ke taur par introduce kiya (Claude ecosystem wali company). Initial release November 2024.
- Original documentation bhi Anthropic ki hai (modelcontextprotocol docs). Python mein ye ek installable package/module hai: `mcp`.

2.2 Analogy 1: USB-C port

Laptop ka USB-C port ek hi standard se monitor, charger, phone sab connect kar deta hai. Isi tarah MCP ek standard se AI app ko Database, GitHub, Files waghera se connect karta hai. **MCP hardware nahi hai, ye sirf analogy hai.**

2.3 Analogy 2: Browser aur Google server (class ki best example)

Tumhare, mere aur kisi aur ke laptop par Chrome hai. Teeno `google.com` kholte hain. Kya Google har browser ke liye alag page banata hai? Nahi. Google ne page **apne server par ek dafa host** kar diya, ab koi bhi browser (client) request bhej kar response (HTML/JSON) le sakta hai.

Traditional web	MCP world
Browser = Client	AI application = MCP Client
Web server (Google server)	MCP Server (jahan tool/resource/prompt host hain)
HTTP / HTTPS request	STDIO ya Streamable HTTP (JSON-RPC messages)
Response = HTML / JSON	Response = tool result / resource data / prompt
Ek dafa host, koi bhi browser use kare	Ek dafa server banao, koi bhi AI app use kare

2.4 With MCP picture

(Upar figure ka right side dekho.) AI app ek **MCP Client** ki tarah behave karta hai. Database, GitHub, Calendar ka code ek **MCP Server** par tools ki surat mein expose hota hai. Naya AI app (App 3) aaye to usay custom integration likhne ki zaroorat nahi, bas wo bhi MCP Client bane aur wahi servers call kare.

> **Sabse common galatfehmi**
>
> MCP **APIs, LLMs, databases ya agents ko replace nahi karta.** Database ka integration code, GitHub ka code, sab pehle jaisa rehta hai. Badalta sirf ye hai ke **app aur external system ke darmiyan connectivity ka tareeqa standard ho jata hai.**

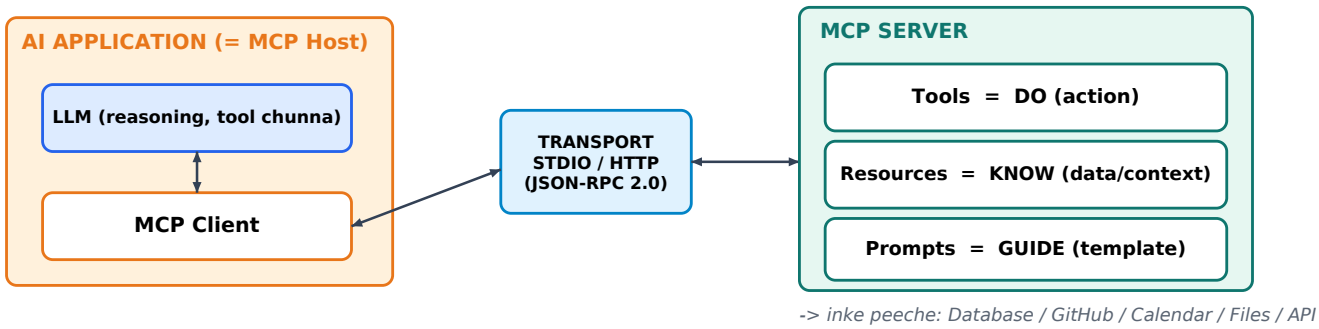
Part 3: Host, Client, Server aur Packages

3.1 Terminology

Term	Matlab	Example
MCP Host	Poori AI application jo user use karta hai. Ye MCP Client use kar ke servers se connect hoti hai.	Claude Desktop, Cursor, GitHub Copilot, aap ki custom app
MCP Client	Host ke andar wo hissa jo server se baat karta hai: tools discover karna, call karna, result wapas lana.	AI app ka client code
MCP Server	Wo process jo external system ko Tools / Resources / Prompts ki surat mein expose karta hai.	Weather MCP Server, DB MCP Server

Browser analogy: Laptop = Host, Browser = Client, Google server = Server. Class mein sir ne kaha: Host ki term zyada important nahi, bas ye samajh lo ke **request kahan se ja rahi hai (client) aur jawab kaun de raha hai (server).** Interview mein 'AI app = MCP client' keh dena kaafi hota hai, host ka zikr bonus hai.

3.2 Architecture ek nazar mein



-> inke peeche: Database / GitHub / Calendar / Files / API

3.3 Ek client, kai servers

Ek MCP Client ek se zyada servers se connect ho sakta hai (jaise Google Drive MCP Server + Weather MCP Server). Har server ke apne alag tools hote hain. LLM user ke sawal ko dekh kar **khud decide karta hai** ke kaun sa server aur kaun sa tool chalana hai, developer ko manual if-else nahi likhna parta.

3.4 Packages ka farq (interview mein zaroor poocha jata hai)

Naam	Kya hai	Kis ne banaya	Kab use karein
MCP (protocol)	Standard/spec jo batata hai AI app aur server kaise baat karein	Anthropic	Concept samajhne ke liye
mcp (Python SDK)	Official package. Isi se MCP client aur server dono ban sakte hain	Anthropic	Core, sab kuch isi se possible
FastMCP	Alag higher-level wrapper/framework, mcp ke upar. Thora convenient, kuch cheezen abstract	Jeremiah Lowin (PrefectHQ), repo: PrefectHQ/fastmcp	Jab jaldi aur asaan server banana ho (industry mein bhi use hota hai)
langchain-mcp-adapters	LangChain ki MCP integration. Server connectivity ka low-level code chhupa deta hai	LangChain developers	Jab app LangChain / LangGraph par bani ho

Naam ki ghalti se bacho

Package install mein **langchain-mcp-adapters** (dash ke sath) hai, handwritten notes mein `langchain_mcp_adapter` likha hai. Sahi naam **langchain-mcp-adapters** hai (import: `langchain_mcp_adapters`). Class mein bhi sir ne pehle 'client' kaha phir correct kar ke 'adapter' kaha. Interview mein 'FastMCP = mcp ke upar high-level wrapper' kehna safe hai.

Part 4: MCP Server kya expose karta hai? Tools, Resources, Prompts

Server ka maqsad external system tak connect karna hai. Documentation ke mutabiq server **3 cheezen** expose kar sakta hai. Class ka asaan formula:

Primitive	Ek lafz mein	Matlab	Kaun control karta hai (spec)
Tools	**DO**	Executable function jo koi kaam kare (search flight, send email, DB query)	Model (LLM decide karta hai kab call karna)
Resources	**KNOW**	Aisi information jo app read kar ke model ko context ke taur par de (file, policy, preferences)	Application / host
Prompts	**GUIDE**	Reusable instruction template (plan_vacation, book_flight)	User (explicitly choose karta hai)

Note: 'kaun control karta hai' wali column MCP spec ki taraf se extra detail hai (class mein nahi boli gayi), lekin interview mein kaam aati hai.

4.1 Tools (DO)

Tool ek **executable function** hai. Travel agency AI system mein tools: `search_flight()`, `book_hotel()`, `book_cab()`, `search_restaurant()`. User kahe 'Bangalore se Dubai flight dhundo' to LLM `search_flight` tool select karta hai.

Tool vs MCP (class mein poochha gaya): Tool sirf external functionality expose karne ka tareeqa hai. MCP wo concept hai jahan tool ko **server par host** kar ke, **client ke zariye call** kiya jata hai. Yani MCP tool calling ko standardize karta hai, tool ki jagah nahi leta.

4.2 Resources (KNOW)

Resource wo information hai jo application read kar ke model ko **context** ke taur par deti hai. Har resource ka ek **URI** hota hai:

URI example	Matlab
travel://preferences	User ki travel preferences
company://policy	Company ki policies
calendar://events	Calendar events
weather://...	Weather data (class demo mein ek resource weather ka bhi tha)

- Resource koi bhi file, document, DB data ya API ka result ho sakta hai. Resource ke andar bhi code likha ja sakta hai (jaise vector database se retrieval), technically possible hai.
- **Tool aur Resource mein farq?** Tool = 'kuch karo' (action). Resource = 'kuch parho / jano' (data). Ye ek tarah ki **segregation** hai: heavy coding / actions tools mein, context data resources mein, taake sab kuch tool ke andar mix na ho jaye.
- Class ka example: pichli application ke notes files (deep_agent.txt, langchain.txt, langgraph.txt) basically resources hi thin.

4.3 Prompts (GUIDE)

Prompt ek **reusable instruction template** hai (e.g. *plan vacation*, *book flight*). Prompt ko server par define karne se koi bhi AI system usay server se le sakta hai. Prompt to prompt hi rehta hai, bas ab wo app ke andar nahi, server par rakha hai.

Zomato analogy: khana wahi hai chahe restaurant mein khao ya Zomato se mangwao. Prompt wahi hai chahe app mein likho ya server se lo.

> **Interview trick:** 'Tools = DO, Resources = KNOW, Prompts = GUIDE'. Aur ye bhi kaho ke MCP server sirf tools tak mehdood nahi, usay 'complete MCP server' tab kehte hain jab tools ke sath resources/prompts bhi expose hon.

Part 5: Transport aur Message Format

MCP mein 2 concepts alag samajhna zaroori hai: **Message format** (kya bheja ja raha hai) aur **Transport** (kaise bheja ja raha hai).

5.1 Message format: JSON-RPC 2.0

Client aur server ke darmiyan har message JSON-RPC 2.0 format mein hota hai. Request mein `method` aur `params`, response mein `result` (ya `error`) hota hai, aur `id` se request-response match hote hain.

```
// Request (client -> server)
{ "jsonrpc": "2.0", "id": 1, "method": "tools/call",
  "params": { "name": "get_current_weather", "arguments": { "city": "Delhi" } } }

// Response (server -> client)  : response ki id request ki id ke barabar hoti hai
{ "jsonrpc": "2.0", "id": 1,
  "result": { "content": [ { "type": "text", "text": "38C, Sunny" } ] } }
```

Common methods (spec se, extra): `tools/list`, `tools/call`, `resources/list`, `resources/read`, `prompts/list`, `prompts/get`. Connection ke shuru mein client aur server ek `initialize` handshake karte hain (protocol version, capabilities). Class demo mein jo 'protocol version' aur 'server info' print hui wo isi handshake ka hissa hai.

5.2 Transport kya hai?

Transport client aur server ke beech **communication ka medium** hai. Do basic transports:

	STDIO	**Streamable HTTP / HTTPS**
Server kahan	**Local**, apni hi machine par	**Remote**, kisi aur ka server (e.g. AWS par)
Kaise baat	Client server process ko khud start karta hai, stdin/stdout par JSON-RPC messages	Network par HTTP requests (internet, URL)
Internet / URL / web server?	Nahi chahiye	Chahiye
Server alag se run karna?	**Nahi**, client khud launch karta hai	Haan, server pehle se hosted chalta hai
Example	Filesystem, Git, Postgres local server	Tavily hosted server, company ka AWS par server
Auth	Process ke environment variables (API key)	API key / OAuth / token

Documentation ke mutabiq aur bhi transports hain (sir ne next class mein dikhane ka kaha), lekin ye 2 basic aur important hain. Purane SSE transport ko Streamable HTTP ne replace kiya hai (extra info).

> **Purane notes ki ghalti theek ki**
>
> Purane notes mein likha tha ke '*MCP Client stdout se request bhejta hai, server stdin se response*'. Ye ulta hai. Sahi: **client server ke stdin mein likhta hai (request), aur server apne stdout par jawab deta hai (response)**. Isko is tarah yaad rakho: stdin = server ka input, stdout = server ka output.

5.3 Local, Remote, Custom servers

- **Local server:** usi machine par chalta hai jahan AI app hai. STDIO use hota hai.
- **Remote server:** kisi aur machine/cloud par. Misal: XYZ company ne tools code kiye aur AWS server par host kiye; apka Claude Code / GitHub Copilot MCP client ban kar HTTP/HTTPS par connect karta hai.
- **Custom server:** jab ready-made server na ho to khud banao (e.g. Weather MCP Server).

Part 6: Code Walkthrough (class ka demo)

Class mein 3 files banayi gayi: **server.py** (tools + resources + prompts), **client.py** (simple MCP client, bina LLM), **client2.py** (agent jo MCP tools use karta hai). Neeche code concept dikhane ke liye hai; SDK versions ke hisaab se class names thori badal sakti hain (class mein latest SDK ka server class istemal hua, purani SDK mein `FastMCP`), decorators wahi rehte hain.

6.1 server.py: tool, resource, prompt expose karna

```
from mcp.server.fastmcp import FastMCP     # latest SDK mein MCP server class bhi use hoti hai

mcp = FastMCP("travel-server")             # variable ka naam kuch bhi ho sakta hai

@mcp.tool()                                 # DO
def search_flight(origin: str, destination: str) -> str:
    return f"Flights {origin} -> {destination}: AI203 08:10, EK565 14:45"

@mcp.tool()
def book_hotel(city: str, nights: int) -> str:
    return f"Hotel booked in {city} for {nights} nights"

@mcp.resource("travel://preferences")       # KNOW  (URI zaroori hai)
def preferences() -> str:
    return "Window seat, vegetarian meals, budget hotels"

@mcp.prompt()                               # GUIDE
def plan_vacation(city: str) -> str:
    return f"Plan a 3-day vacation in {city} with flights, hotel and food."

if __name__ == "__main__":
    mcp.run(transport="stdio")              # remote ke liye: transport="streamable-http"
```

- **Decorator hi batata hai** ke function tool hai, resource hai ya prompt. Agar variable ka naam `mcp` hai to `@mcp.tool()`, agar `server` hai to `@server.tool()`. Ye sirf variable ka naam hai, magic nahi.
- Resource mein **URI** dena parta hai (`travel://preferences`), taake client usay pehchan kar read kare.

6.2 client.py: raw MCP client (bina LLM)

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

params = StdioServerParameters(command="python", args=["server.py"])  # server ka path

async def main():
    async with stdio_client(params) as (read, write):      # client server ko khud start karta hai
        async with ClientSession(read, write) as session:
            await session.initialize()                      # handshake
            tools = await session.list_tools()              # tools/list
            print([t.name for t in tools.tools])
            res = await session.call_tool("search_flight",
                      {"origin": "BLR", "destination": "DXB"})   # tools/call
            print(res.content)
            r = await session.read_resource("travel://preferences")
            print(r.contents)
            print(await session.list_prompts())
asyncio.run(main())
```

Class ka famous sawal

Server.py ko alag se run kyun nahi kiya? Kyunke transport STDIO tha aur dono local hain. STDIO mein **client khud server process ko launch kar leta hai** (command + args ke zariye). Remote HTTP server hota to wo pehle se hosted chal raha hota aur client sirf URL se connect karta.

6.3 client2.py: Agent + MCP tools (LangChain adapter)

Yahan agent ko tools **server se** milte hain, isi file mein define nahi hote. LLM khud decide karta hai kaun sa tool chalana hai.

```
import asyncio
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

client = MultiServerMCPClient({
    "travel": {"transport": "stdio", "command": "python", "args": ["server.py"]},
    # remote: {"transport": "streamable_http", "url": "https://<host>/mcp"},
})

async def main():
    tools = await client.get_tools()                # tools MCP server se aaye
    agent = create_agent(ChatOpenAI(model="gpt-4o-mini"), tools)
    out = await agent.ainvoke({"messages": [("user", "Find flight from BLR to Dubai")]})
    print(out["messages"][-1].content)
asyncio.run(main())
```

Agentic flow with MCP tools



1) tools/list se tools milte hain 2) LLM decide karta hai 3) tools/call 4) result LLM ko wapas 5) LLM final answer banata hai

6.4 Async kyun?

LLM call, API call aur server response sab **I/O wait** wali operations hain. Bina async ke program ek call ke jawab ka intezar karta rahta hai aur baqi kaam ruk jate hain. Async mein ek task wait kar raha ho to doosra chalta rehta hai, is se program block nahi hota. Isi liye MCP client code `async/await` mein hota hai.

6.5 Class mein aaya error aur fix

Agent chalate waqt model ne error diya (reasoning aur function tools ek sath support nahi). Fix: `ChatOpenAI` ke `reasoning_effort` parameter ko None rakha, phir chal gaya. Ye MCP ka issue nahi tha, model-call ka issue tha. Import error aya to `mcp[cli]` aur LangChain MCP adapter packages install karne se theek hua.

6.6 Practical project se: 3 tarah ke servers ek hi client mein (TripMate AI)

```
pip install langchain-mcp-adapters nest-asyncio mcp

client = MultiServerMCPClient({
  "tavily": {                                  # REMOTE
    "transport": "streamable_http",
    "url": "https://mcp.tavily.com/mcp/?tavilyApiKey=<KEY>"},
  "aviationstack": {                           # LOCAL (open-source)
    "transport": "stdio", "command": "uvx", "args": ["aviation-stack-mcp"],
    "env": {"AVIATIONSTACK_API_KEY": "<KEY>"}},
  "weather": {                                 # CUSTOM
    "transport": "stdio", "command": "python",
    "args": ["custom_weather_mcp_server.py"],
    "env": {"OPENWEATHER_API_KEY": "<KEY>"}},
})
tools = await client.get_tools()          # sab servers ke tools ek list mein
```

Debug tip: `asyncio.run() cannot be called from a running event loop` error aaye (FastAPI + MCP dono async) to start par `nest_asyncio.apply()` chalao. Tool call fail ho to pehle `client.get_tools()` print kar ke tool ka naam match karo.

Part 7: Real-world baaten (doubt session se)

7.1 Name conflict: local tool aur MCP tool ka same naam

Agar agent ke paas local `search_flight` bhi ho aur MCP server ka bhi `search_flight`, to behavior ambiguous ho jata hai aur information override ho sakti hai. Ye **bad design** hai. Fix: **prefix / namespace** use karo, jaise `local_search_flight` aur `mcp_search_flight`. Agent ko MCP tools dene ka matlab hai LLM ko un tools ki list pata chal jati hai, aur wahi decide karta hai kaun sa chalana hai.

7.2 Scaling: 10,000 tools ek server par nahi

Agar hazaron tools hon to unhein ek hi MCP server par host nahi karte. Design: **MCP gateway** banao aur tools ko category ke hisaab se alag servers mein baanto (Finance MCP server, HR MCP server, Marketing MCP server). Gateway se requests route hoti hain. Ye system-level design ka masla hai: load, rate limit, monitoring, caching sab sochna parta hai. Cloud par host karna bhi possible hai.

7.3 Vendor / third-party MCP servers

- Bohat si SaaS companies apna **official hosted MCP server** deti hain: Notion, Databricks, Cloudflare, Atlassian (Jira), Microsoft waghera. Connect karne ke liye aksar **MCP URL + auth (OAuth/access token)** chahiye.
- **Do raste:** (a) vendor ka official MCP endpoint use karo, (b) agar wo na de to apna server banao jisme us vendor ki API ka connectivity code likho (jaise class mein server.py mein).
- Vendor aap ko sirf apna managed server use karne par majboor kar sakta hai (security/policy ki wajah se). Enterprise mein security aur access control ki wajah se log aksar vendor-hosted servers ko prefer karte hain. Ye vendor ka faisla hai.
- Agar `mcp` package direct use karo to client.py mein connectivity khud likhni parti hai. **LangChain adapter wahi connectivity ka low-level code chhupa deta hai**, tum sirf server path/URL dete ho (abstraction layer).

7.4 Existing apps mein MCP

GitHub Copilot ko **Agent mode** mein rakh kar MCP server configure karo to wo MCP tools (GitHub, Databricks, Notion ya apni services) use kar sakta hai. Enterprise mein aksar admin ye pehle se configure kar deta hai. Claude Code, Cursor bhi MCP clients hain. Naye project mein project-level

instructions .md file (jaise CLAUDE.md) aur MCP config (mcp.json) saath rakhna ek acha tareeqa hai.

7.5 Security best practices

- API keys code mein hardcode na karo, `.env` / environment variables use karo.
- Remote server par hamesha **HTTPS** aur authentication (API key / OAuth) confirm karo.
- Custom server mein tool arguments par **input validation** lagao.
- Production mein rate limiting aur logging lagao.
- Sirf wahi data expose karo jo zaroori hai. Class mein kaha gaya: enterprise mein system managed aur secure tareeqe se design hota hai, LLM ko jo data jata hai wo control mein hota hai.

7.6 A2A (Agent-to-Agent)

Sawal aya ke kya A2A MCP ko replace kar raha hai. **Nahi.** A2A basically **agent-to-agent communication protocol** hai. Sir ne ise MCP ke baad alag se cover karne ka kaha. Rough farq: **MCP = agent/app ko tools aur data se jorna**, **A2A = agents ko aapas mein baat karwana**. Dono complementary hain (ye summary extra hai).

Part 8: MCP vs normal function calling

Aspect	Normal tool / function calling	MCP
Tool kahan rehta hai	App ke andar hi, usi codebase mein	Alag, reusable MCP server mein
Framework switch (LangGraph -> CrewAI)	Har framework ke liye tool dobara likhna	Server wahi rehta hai, sirf client side adapter badalta hai
Naya AI app aaye	Naye app mein dobara integration	Bas MCP client bano aur server se connect karo
API v1 -> v2	Har app mein code update	Sirf server mein update; app ka code nahi badalta
Sharing	Code copy-paste	Server URL ya package share karo
Maintenance	Har API ki doc khud dekhna	Third-party/vendor server khud maintain karta hai
Scale	Hazaron tools manage karna mushkil	Reusable servers, categories mein baante ja sakte hain

> **Interview mein overclaim mat karo**
>
> Purane notes mein 'self-healing, koi downtime nahi' likha tha. Ye **overstated** hai. Sahi baat: API change ka asar **server tak mehdood** rehta hai aur client apps ko code nahi badalna parta, lekin **kisi ko server update karna parta hai** (aap ya vendor). MCP magically downtime khatam nahi karta.

MCP kab use karein / kab nahi

- **Karein:** agent ko external tools/APIs/DB se baat karni ho, multiple apps ya frameworks ek hi tool use karein, API frequently change hoti ho, production-grade maintainable system chahiye.
- **Zaroorat nahi:** sirf plain LLM chat ho aur koi external tool use hi na ho. Chhoti single-app prototype mein ek simple function tool kaafi hai.

Part 9: Interview Q&A

Q1. MCP kya hai?

Model Context Protocol ek open standard protocol hai (Anthropic ne introduce kiya) jo AI applications ko external tools, data aur systems se standard tareeqe se connect karta hai. Analogy: AI apps ke liye USB-C port.

Q2. MCP ki zaroorat kyun pari?

Bina MCP ke har AI app ko har external system ke liye custom integration likhni parti hai (N x M), framework lock-in hota hai, aur API change par har app break hoti hai. MCP tool calling ko standardize karta hai.

Q3. MCP architecture batao.

Host (AI app) ke andar MCP Client hota hai jo MCP Server se transport (STDIO/Streamable HTTP) par JSON-RPC 2.0 messages exchange karta hai. Server Tools, Resources, Prompts expose karta hai aur peeche external system se connect hota hai.

Q4. Tools, Resources aur Prompts mein farq?

Tools = DO (executable actions), Resources = KNOW (context data jo URI se read hota hai), Prompts = GUIDE (reusable instruction templates).

Q5. STDIO aur Streamable HTTP mein kab kya?

STDIO local server ke liye: client server process launch karta hai, stdin/stdout par messages, internet nahi chahiye. Streamable HTTP remote/cloud server ke liye: network par HTTP, URL aur auth chahiye.

Q6. MCP kaun sa message format use karta hai?

JSON-RPC 2.0: request mein jsonrpc, id, method (e.g. tools/call), params; response mein id aur result ya error.

Q7. MCP, FastMCP aur langchain-mcp-adapters mein farq?

MCP = protocol + official Python SDK (Anthropic). FastMCP = higher-level wrapper (Jeremiah Lowin / PrefectHQ). langchain-mcp-adapters = LangChain ki integration jo MCP servers ko LangChain tools banati hai aur connectivity chhupati hai.

Q8. MCP aur function calling mein kya farq hai?

Function calling mein tool app ke andar hota hai aur framework-specific hota hai. MCP mein tool alag reusable server par hota hai aur kisi bhi MCP client se use ho sakta hai. MCP function calling ko replace nahi karta, standardize karta hai.

Q9. Kya MCP APIs ya agents ko replace karta hai?

Nahi. APIs, LLMs, databases aur agents pehle jaise rehte hain. MCP sirf unke darmiyan connectivity standard karta hai.

Q10. Tool ko LLM kaise chunta hai?

Client tools/list se tools ki list (naam, description, schema) leta hai aur LLM ko deta hai. LLM user query ke mutabiq tool select karta hai, client tools/call bhejta hai, result LLM ko wapas jata hai.

Q11. MCP client code async kyun hota hai?

Server/API/LLM calls I/O wait wale hote hain. Async se ek call ka wait karte hue baqi kaam block nahi hote.

Q12. Server.py ko alag se kyun nahi chalaya (STDIO)?

STDIO mein client command/args ke zariye server ko subprocess ki tarah khud launch karta hai. Sirf remote HTTP server pehle se hosted hota hai.

Q13. Local aur MCP tool ka naam same ho to?

Ambiguity aur override ka risk. Namespace prefix use karo (local_ / mcp_).

Q14. Hazaron tools ho to kya karoge?

Ek server par nahi. MCP gateway + category-wise servers (Finance, HR, Marketing), plus rate limiting, logging aur monitoring.

Q15. Vendor MCP servers (Notion, Databricks) kaise use karte hain?

Official hosted MCP endpoint (URL + OAuth/token) se connect karo. Agar wo na ho to apna MCP server bana kar us vendor ki API ka connectivity code likho. Vendor policy ki wajah se sirf apna managed server allow kar sakta hai.

Q16. MCP security ke liye kya karoge?

Secrets env vars mein, HTTPS + auth, tool-argument validation, least-privilege data exposure, rate limiting aur logging.

Q17. MCP Host aur MCP Client mein farq?

Host = poori AI application jise user use karta hai (Claude Desktop, Cursor). Client = host ke andar wo component jo server se connect hota hai. Practically 'AI app = MCP client' keh dena kaafi hai.

Q18. A2A kya hai, kya MCP ko replace karta hai?

A2A agent-to-agent communication protocol hai. MCP agent ko tools/data se jorta hai, A2A agents ko aapas mein. Replace nahi, complement.

Cheat sheet: 30 second revision

Cheez	Yaad rakho
Full form	Model Context Protocol (Anthropic, open source, Nov 2024)
Problem	N x M custom integrations, lock-in, API change break
Roles	Host (AI app) > MCP Client <-> MCP Server > external system
Server exposes	Tools (DO), Resources (KNOW, URI), Prompts (GUIDE)
Message format	JSON-RPC 2.0 (tools/list, tools/call, resources/read, prompts/get)
Transport	STDIO = local, Streamable HTTP/HTTPS = remote
Packages	mcp (official SDK), FastMCP (wrapper), langchain-mcp-adapters (LangChain)
Decorators	@mcp.tool(), @mcp.resource('uri://...'), @mcp.prompt()
Async	I/O wait par program block na ho
Not replaced	APIs, LLMs, databases, agents (sab pehle jaise)
Scale	Gateway + category-wise servers

> **Interview-ready one-liner:** '*MCP ek open standard protocol hai jo AI applications (LLMs/agents) ko external tools, APIs aur data sources se standard tareeqe se connect karta hai: MCP Client aur MCP Server ke beech JSON-RPC 2.0 messages, STDIO (local) ya Streamable HTTP (remote) transport par, aur server Tools, Resources aur Prompts expose karta hai.*'

Source: class transcript (11 Oct 2026), handwritten notes aur MCP_Complete_Notes.pdf. 'Extra' likhi hui details MCP spec se add ki gayi hain.